



I have attached the Lab 4 handout and the assigned Zombie Escape project/README. Please go through the lab requirements and the README carefully before making any changes. Identify all the tasks that need to be implemented, the required testing, the Git/commit requirements, and the final deliverables. Also inspect the existing game structure so that future changes can be made with minimum modifications and without unnecessarily changing the existing code.

🔗 https://github.com/SETAPESU26/25_zombie_escape.git

Worked for 21s

Yes. I went through **both the Lab 4 handout and the assigned 25_zombie_escape repository/README**, and I also inspected the existing `game.py` structure before considering any modifications.

The important point is: **we should not rewrite this game**. The existing project is deliberately small and can be extended with targeted changes while preserving its current structure.

1. What Lab 4 actually requires

The handout establishes the overall workflow:

1. Go to the assigned repository.
2. Read the README and understand the deliverables.
3. Clone/fork the repository.
4. Run the Python game.
5. Record **10 seconds before making changes**.
6. Use Vibe Coding prompts to fix/add functionality.
7. Implement the README features **one by one**.
8. Record another **10 seconds after all changes**.
9. Push the code to your personal repository.
10. Have **separate commits for each task**.
11. **Do not create a PR to the SETAPESU26 main repository.**



Lab_4_VibeCoding_Student_handout

The handout's final Lab-4 folder deliverables are:

- Before video
- After video
- Updated code
- Chat history exported as DOC/PDF



Lab_4_VibeCoding_Student_handout

There is one subtlety: the repository README's submission section says the final submission is specifically the **two 10-second videos plus the complete LLM chat-history link**.  GitHub

So I'd follow **both**: keep your updated code in your Lab-4 repository, and make sure your final submission contains the items required by your handout.

2. The four actual coding tasks

The README has exactly **four tasks**.

Task 1 — Health System

Current game:

- Player has no health value.
- One zombie collision immediately sets `game_over = True`.

The required behavior is:

- Player starts with **3 HP**
- Zombie touch removes **1 HP**
- Game ends only when HP becomes **0**
- There must be a short **invincibility window** after being hit
- HP must appear in the HUD.  GitHub

Testing

You need to demonstrate:

Zombie touches player → HP decreases → player survives.

Then another hit should decrease HP again.

During the invincibility period, another collision should **not** immediately remove another HP.

Only when HP reaches 0 should:

```
game_over = True
```

The README explicitly says the game-over screen should appear **only at 0 HP**.  GitHub

3. Task 2 — Ammo System

Current game has effectively unlimited shooting, apart from the 15-frame shooting cooldown.

Required:

- Maximum **12 bullets per clip**
- Shooting stops when ammo reaches 0
- Reload automatically starts
- Reload takes **2 seconds**
- Ammo is restored only after the complete 2 seconds
- Ammo count must appear in HUD.  [GitHub](#)

Testing

You need to:

1. Shoot repeatedly.
2. Reach 0 ammo.
3. Verify another click doesn't fire.
4. Observe reload timer.
5. Wait the complete 2 seconds.
6. Verify ammo becomes 12.

The README specifically requires checking that the reload timer counts down correctly and that ammo isn't restored prematurely.  [GitHub](#)

4. Task 3 — Explosive Barrels

Required:

- Put **4 barrels** on the map.
- Shoot a barrel.
- It explodes.
- Zombies within a fixed radius are destroyed.
- Barrel disappears after explosion.  [GitHub](#)

Testing

The required test is very specific:

Shoot a barrel next to a group of zombies.

You should be able to visibly show:

Bullet → Barrel → Explosion → Nearby zombies removed → Barrel gone

This will make an excellent part of the after-change video.

5. Task 4 — Zombie Types

There are currently only one type of zombie:

Plain text

```
Zombie
speed = 1.5
HP = 3
size = 30 × 30
```

The README requires two additional types:

Fast Zombie

- Higher speed
- 1 HP
- Smaller than standard zombie

Tank Zombie

- Lower speed
- 6 HP
- Larger than standard zombie

Both must be mixed into normal wave spawning.  [GitHub](#)

Testing must prove:

- Fast zombies actually spawn.
 - Tank zombies actually spawn.
 - Their different speeds are noticeable.
 - Their different HP values affect combat.
 - Their different sizes are visible.
 - Existing wave progression still works.
-

6. Existing code structure

This is actually good news.

The repository contains only:

</> Plain text

```
.gitignore  
README.md  
game.py
```

and the game itself is only about **184 lines** of Python.  [GitHub](#) +1

So do not introduce a complicated architecture.

The existing structure is:

</> Plain text

```
game.py  
|  
├─ Constants  
|   ├── WIDTH / HEIGHT  
|   ├── FPS  
|   └─ BG  
|  
├─ Zombie  
|   ├── __init__()  
|   ├── update()  
|   ├── hit()  
|   └─ draw()  
|  
├─ spawn_zombie()  
|  
├─ Player  
|   ├── move()  
|   ├── shoot()  
|   ├── update_bullets()  
|   └─ draw()  
|
```

The repository currently has exactly this simple architecture.  [GitHub](#)

7. Where each feature should go

This is the most important part for our future prompts.

We should make **minimum modifications**.

Task 1

Primarily modify:

</> Plain text

```
Player
GameEngine.reset()
GameEngine.update()
GameEngine.draw()
```

The current collision logic is here:

</> Plain text

```
GameEngine.update()
↓
for z in self.zombies
↓
z.update()
↓
if z.rect.colliderect(self.player.rect)
↓
self.game_over = True
```

That is the exact place where the health/invincibility logic belongs. [⊕ GitHub](#)

We shouldn't create a completely new collision architecture.

Task 2

Primarily modify:

</> Plain text

```
Player.__init__()
Player.shoot()
```

```
Player.move()
```

and probably:

</> Plain text

```
GameEngine.update()  
GameEngine.draw()
```

The existing shooting pipeline is already nicely isolated:

</> Plain text

```
Mouse click  
  ↓  
handle_events()  
  ↓  
player.shoot()  
  ↓  
player.bullets  
  ↓  
update_bullets()  
  ↓  
collision detection
```

So we can add ammunition to this existing pipeline rather than redesigning shooting. [GitHub](#)

Task 3

This can be added with a small new object, for example:

</> Plain text

```
Barrel
```

and a small amount of logic in:

</> Plain text

```
GameEngine.reset()  
GameEngine.update()
```

```
GameEngine.draw()
```

We don't need a new subsystem.

The existing bullets are already represented as moving `[x, y, vx, vy]` objects, and bullet collision is already handled in `GameEngine.update()` . [GitHub](#)

So barrels can naturally plug into that existing collision mechanism.

Task 4

This is where we should be particularly careful.

Instead of rewriting `Zombie` , we can preserve the existing class and introduce the minimum required specialization.

Conceptually:

`</>` Plain text

```
Zombie
|
├── FastZombie
|
└── TankZombie
```

or use a controlled type parameter if that results in fewer modifications.

The important thing is that existing code such as:

`</>` Plain text

```
z.update(...)
z.hit()
z.draw(...)
z.rect
z.hp
```

should continue to work.

That means `GameEngine` doesn't need a completely different zombie system.

8. Existing game flow

I traced the current flow because the README specifically tells the student to understand it before modifying the code.  GitHub

It currently works approximately like this:

 Plain text

```
GameEngine.run()
|
▼
handle_events()
|
|— W/A/S/D → Player.move()
|
|— Mouse → Player.shoot()
|
|— R → reset()
|
▼
update()
|
|— Player movement
|— Bullet movement
|— Score update
|— Zombie movement
|— Player/Zombie collision
|— Bullet/Zombie collision
|— Kill counting
```

That is a very clean insertion point for all four tasks.

9. One existing oddity I noticed

There is a slightly unusual implementation in `Player.shoot()` :

 Plain text

```
self.bullets.append(pygame.Rect(...))
self.bullets.append([x, y, vx, vy])
self.bullets.pop(-2)
```

So the temporary `pygame.Rect` is immediately removed and the actual bullet is stored as:

`</>` Plain text

```
[x, y, vx, vy]
```

The rest of the game expects that representation. [GitHub](#)

We should not "clean this up" just because it looks unusual.

That would violate the spirit of:

"Do not replace the whole project with an unrelated implementation."

The README explicitly says to inspect the existing code, test generated changes, keep the complete chat history, and **not replace the whole project with an unrelated implementation.** [GitHub](#)

So unless something is actually broken, we'll leave it alone.

10. Dependency requirement

The README explicitly says:

No unnecessary external dependencies beyond pygame. [GitHub](#)

Therefore our implementation should stay with:

`</>` Python

```
pygame
random
math
time
```

and Python standard library functionality.

No external health library, no physics library, no new game framework, etc.

The README also says the game uses `pygame-ce` for Python 3.12+, and can be started with:

`</>` Bash

```
pip install pygame-ce
python game.py
``` :chatgpt-content-reference{index="18"}
```

---

# 11. Commit strategy

This is important because your instructor specifically requires **\*\*separate commits for each task\*\***. :chatgpt

I recommend exactly:

```
```text
Commit 1:
Task 1 - Add player health and invincibility

Commit 2:
Task 2 - Add ammo and reload system
```

Then push all four commits to **your personal fork/repository**.

Do **not** squash them into one commit.

Do **not** raise a PR against:

</> Plain text

SETAPESU26/25_zombie_escape

The handout explicitly prohibits that.  Lab_4_VibeCoding_Student_handout

12. Testing plan

We should test after **each task**, rather than implementing all four and hoping everything works.

Before Task 1

Run the original game.

Record:

10-second BEFORE video

Show normal gameplay.

The handout explicitly requires this recording **before any changes**.  Lab_4_VibeCoding_Student_handout

After Task 1

Test:

- 3 HP
- hit once → 2 HP
- hit again → 1 HP
- hit during invincibility → no extra damage
- hit after invincibility → HP decreases
- 0 HP → game over
- R → restart

Then commit.

After Task 2

Test:

- Start with 12 bullets.
- Shoot.
- Ammo decreases.
- Reach 0.
- Shooting is blocked.
- Reload begins.
- Timer counts down.
- After 2 seconds → 12 bullets.
- Repeat.

Then commit.

After Task 3

Test:

- Four barrels visible.
- Shoot barrel.

- Explosion happens.
- Nearby zombies disappear.
- Barrel disappears.
- Zombies outside radius survive.

Then commit.

After Task 4

Test:

- Standard zombie still works.
- Fast zombie appears.
- Fast zombie has 1 HP.
- Fast zombie is smaller/faster.
- Tank zombie appears.
- Tank zombie has 6 HP.
- Tank zombie is larger/slower.
- Waves continue progressing.

Then commit.

13. Final integrated testing

The README's required testing combines everything:

survive several waves, take hits, verify health/invincibility, empty and reload a clip, shoot a barrel beside zombies, verify both zombie types, confirm game over only at 0 HP, and restart after game over.  [GitHub](#)

So after Task 4, we should do one final complete run.

That run is what I'd use for the **10-second AFTER video**.

Ideally the 10 seconds should visibly demonstrate as many new features as possible rather than just showing the game running.

14. Chat history requirement

This is particularly relevant because you're doing a Vibe Coding assignment.

The README says:

- Keep the **complete LLM chat history**
- Include the complete chat-history link
- The goal is to use the LLM as a coding assistant while **you retain responsibility for understanding and testing.**  GitHub

The handout additionally says the chat history should be exported as a **DOC/PDF** for the Lab-4 folder.

 Lab_4_VibeCoding_Student_handout

So we should **not delete the prompts/conversation after we're done.**

15. Most important: how I suggest we proceed

Since this is specifically a **Vibe Coding** lab, I would **not jump straight to writing the complete modified game.**

Instead, we should follow the assignment's intended workflow:

 Plain text

ORIGINAL CODE

|

▼

Understand structure

|

▼

Task 1 prompt

|

▼

Test

|

▼

Commit 1

|

▼

Task 2 prompt

|

▼

Test

|

▼

And importantly, **each prompt should be focused enough that the AI modifies the existing code rather than replacing it.**

The README itself says to inspect existing code before asking for changes and to avoid replacing the whole project.  GitHub

My recommended order

Task 1 → Health

Task 2 → Ammo

Task 3 → Barrels

Task 4 → Zombie types

That order minimizes conflicts because health and ammo primarily modify `Player / GameEngine`, barrels add a new entity, and zombie specialization comes last.

So at this point, **we have the requirements and the existing architecture mapped out. No code should be changed yet.**

The next step should be to create the **first Vibe Coding prompt for Task 1**, deliberately written so the AI makes the smallest possible changes to the existing `game.py` and explains what it changed.

Implement Task 1 only in the existing Zombie Escape game. Give the player 3 HP. One zombie touch should subtract 1 HP. The game should end only when HP reaches 0. Add a brief invincibility window after each hit to prevent instant repeated damage, make the invincibility visible, and display HP in the HUD. Keep the existing code structure and make only the necessary changes. Do not modify features that are not part of Task 1. `game.py` file go through it and tell me where to change dont give me entire code i will only change

Yes. I went through the **actual `game.py`** from the assigned repository. For Task 1, you only need to make changes in **four places**. Do **not** replace the whole file.

The current collision that immediately ends the game is here: `GameEngine.update()`, lines 124–127.



1. Add HP and invincibility to `Player.__init__()`

Find this section around lines 51–56:

 Python

```
class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60,160,220)
        self.bullets = []
        self.shoot_cooldown = 0
```

Add these **two variables** at the end:

 Python

```
self.hp = 3
self.invincibility_timer = 0
```

So you are only adding:

 Python

```
self.hp = 3
self.invincibility_timer = 0
```

Do not change anything else in `Player.__init__()` .  [GitHub](#)

2. Decrease the invincibility timer

Go to `Player.move()` around lines 57–66.

You currently have:

 Python

```
if self.shoot_cooldown > 0:
    self.shoot_cooldown -= 1
```

Immediately after that, add:

 Python

```
if self.invincibility_timer > 0:  
    self.invincibility_timer -= 1
```

So it becomes:

 Python

```
if self.shoot_cooldown > 0:  
    self.shoot_cooldown -= 1  
if self.invincibility_timer > 0:  
    self.invincibility_timer -= 1
```

We're using frames here because the game already runs at:

 Python

```
FPS = 60
```

So, for example, if we later set the timer to `60` , that's approximately **1 second** of invincibility.  [GitHub](#)

I'd recommend **60 frames / 1 second** for this lab.

3. Replace **ONLY** the zombie collision logic

This is the most important change.

Currently around lines **124–127** you have:

 Python

```
for z in self.zombies:  
    z.update(self.player.rect.center)  
    if z.rect.colliderect(self.player.rect):  
        self.game_over = True
```

Do **not** change the zombie update.

Replace only the collision part:

 Python

```
if z.rect.colliderect(self.player.rect):
    self.game_over = True
```

with:

</> Python

```
if z.rect.colliderect(self.player.rect):
    if self.player.invincibility_timer <= 0:
        self.player.hp -= 1
        self.player.invincibility_timer = 60

    if self.player.hp <= 0:
        self.game_over = True
```

So the complete small section becomes:

</> Python

```
for z in self.zombies:
    z.update(self.player.rect.center)
    if z.rect.colliderect(self.player.rect):
        if self.player.invincibility_timer <= 0:
            self.player.hp -= 1
            self.player.invincibility_timer = 60

    if self.player.hp <= 0:
        self.game_over = True
```

What this does

First touch:

</> Plain text

```
HP = 3
↓
zombie touches
↓
HP = 2
↓
1 second invincibility
```

Another touch during that second:

</> Plain text

ignored

After invincibility:

</> Plain text

zombie touches

↓

HP = 1

Then:

</> Plain text

HP = 0

↓

game_over = True

This directly satisfies Task 1 without changing the existing zombie, shooting, wave, or scoring systems.

4. Display HP in the HUD

Go to `GameEngine.draw()` .

Currently around lines **158–161**:

</> Python

```
hud = self.font.render(  
    f"Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.kills_to_next} | WASD Move,  
    True, (160,220,120))
```

Change **only the formatted text** to include HP.

For example:

</> Python

```
hud = self.font.render(  
    f"HP: {self.player.hp} Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.kills_to_win}  
    True, (160,220,120))
```

That's all you need for the HUD. [GitHub](#)

5. Make invincibility visible

This is the one additional change needed because the task explicitly says:

make the invincibility visible.

The cleanest minimal modification is inside `Player.draw()`.

Currently:

`</>` Python

```
def draw(self, screen):  
    pygame.draw.rect(screen, self.color, self.rect, border_radius=6)  
    for b in self.bullets:  
        pygame.draw.circle(screen, (255,220,60), (int(b[0]), int(b[1])), 5)
```

Change the first line to conditionally make the player visually blink during invincibility.

Replace:

`</>` Python

```
pygame.draw.rect(screen, self.color, self.rect, border_radius=6)
```

with:

`</>` Python

```
if self.invincibility_timer <= 0 or self.invincibility_timer % 10 < 5:  
    pygame.draw.rect(screen, self.color, self.rect, border_radius=6)
```

Keep the bullet code exactly as it is:

</> Python

```
for b in self.bullets:
    pygame.draw.circle(screen, (255,220,60), (int(b[0]), int(b[1])), 5)
```

Result

During invincibility the player will **blink**:

</> Plain text

VISIBLE → INVISIBLE → VISIBLE → INVISIBLE

This makes the temporary invincibility clearly visible without adding another graphical system.

So your total changes are only these

Location	Change
Player.__init__()	Add hp = 3
Player.__init__()	Add invincibility_timer = 0
Player.move()	Decrease timer each frame
Player.draw()	Blink player during invincibility
GameEngine.update()	Replace instant game-over collision with HP logic
GameEngine.draw()	Add HP to HUD

That's it.

Do not change:

- Zombie
- spawn_zombie()
- shooting
- bullets
- score

- kills
- waves
- zombie HP
- zombie speed
- controls
- game-over screen
- restart mechanism

Those are outside Task 1.

One important testing point

After making these changes, test specifically:

1. Start → HUD shows **HP: 3**
2. Let one zombie touch you → **HP: 2**
3. Stay in contact with zombie → HP should **not immediately become 1**
4. Wait for the blinking/invincibility period to finish.
5. Get touched again → **HP: 1**
6. Repeat → **HP: 0**
7. Only then should **DEVoured!** appear.
8. Press **R** → game restarts with **HP: 3**.

That last test works automatically because `reset()` creates a new `Player` :

 Python

```
self.player = Player(WIDTH//2, HEIGHT//2)
```

and your new `Player` constructor initializes HP to 3.  [GitHub](#)

Your Task 1 commit

Once those tests pass, make **only this task's commit**, for example:

 Bash

```
git add game.py
git commit -m "Task 1: Add player health and invincibility"
```

Then we should **stop there**. Don't implement Task 2 until Task 1 is tested and committed, because your lab requires separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

Keep Task 1 unchanged and implement Task 2 only. Limit the player to 12 bullets per clip. Each successful shot should decrease the ammo count. Prevent shooting when the clip is empty and trigger a reload that takes exactly 2 seconds before restoring the ammo. Display the ammo count in the HUD. Preserve the existing code structure and do not change the completed Health System.

Absolutely. **Keep all Task 1 changes exactly as they are**. For Task 2, make only the following additions/changes to your current `game.py`.

The original project uses `FPS = 60` and has the shooting logic isolated inside `Player.shoot()`, so we can implement the 12-bullet clip and 2-second reload without restructuring the game.  [GitHub](#)

1. Add ammo variables in `Player.__init__()`

You already added your Task 1 variables here.

You should currently have something similar to:

 Python

```
self.bullets = []
self.shoot_cooldown = 0
self.hp = 3
self.invincibility_timer = 0
```

Add **two Task 2 variables** after those:

 Python

```
self.ammo = 12
self.reload_timer = 0
```

So the relevant part becomes:

 Python

```
self.bullets = []
self.shoot_cooldown = 0
```

```
self.hp = 3
self.invincibility_timer = 0
self.ammo = 12
self.reload_timer = 0
```

Don't change the HP variables.

2. Add reload countdown in `Player.move()`

You already modified `Player.move()` for Task 1.

You should have something like:

`</>` Python

```
if self.shoot_cooldown > 0:
    self.shoot_cooldown -= 1

if self.invincibility_timer > 0:
    self.invincibility_timer -= 1
```

Add the reload logic **after that**:

`</>` Python

```
if self.reload_timer > 0:
    self.reload_timer -= 1
    if self.reload_timer == 0:
        self.ammo = 12
```

So:

`</>` Python

```
if self.shoot_cooldown > 0:
    self.shoot_cooldown -= 1

if self.invincibility_timer > 0:
    self.invincibility_timer -= 1

if self.reload_timer > 0:
    self.reload_timer -= 1
```

```
if self.reload_timer == 0:  
    self.ammo = 12
```

Why 120 frames?

Your game already has:

</> Python

```
FPS = 60
```

Therefore:

</> Plain text

```
60 frames = 1 second  
120 frames = 2 seconds
```

So we will use:

</> Python

```
self.reload_timer = 120
```

3. Modify `Player.shoot()`

This is the main Task 2 change.

Your current shooting method starts with:

</> Python

```
def shoot(self, target_pos):  
    if self.shoot_cooldown > 0: return
```

Change that line to:

</> Python

```
def shoot(self, target_pos):
    if self.shoot_cooldown > 0 or self.reload_timer > 0 or self.ammo <= 0:
        return
```

This prevents shooting when:

- normal shooting cooldown is active
- reload is happening
- ammo is zero

4. Decrease ammo only after a successful shot

Your existing code eventually does:

 Python

```
self.bullets.append(pygame.Rect(cx-4, cy-4, 8, 8))
self.bullets.append([cx-4, cy-4, vx, vy])
self.bullets.pop(-2)
self.shoot_cooldown = 15
```

Change only the end to:

 Python

```
self.bullets.append(pygame.Rect(cx-4, cy-4, 8, 8))
self.bullets.append([cx-4, cy-4, vx, vy])
self.bullets.pop(-2)

self.ammo -= 1
self.shoot_cooldown = 15

if self.ammo == 0:
    self.reload_timer = 120
```

So the important sequence is:

 Plain text

```
Successful shot
↓
ammo -= 1
↓
```

```
ammo becomes 0?  
↓  
start 120-frame reload
```

This is important because the ammo should **not decrease** if the player clicks but doesn't actually fire.

For example, this existing check:

```
</> Python  
  
if dist == 0: return
```

remains untouched.

5. Don't change the Health System

Your Task 1 code inside `Player.shoot()` should obviously not exist, but more importantly:

Do not touch these Task 1 variables:

```
</> Python  
  
self.hp  
self.invincibility_timer
```

And don't modify your zombie collision code.

Your Task 1 collision should remain exactly as you implemented it:

```
</> Python  
  
if z.rect.colliderect(self.player.rect):  
    if self.player.invincibility_timer <= 0:  
        self.player.hp -= 1  
        self.player.invincibility_timer = 60  
  
    if self.player.hp <= 0:  
        self.game_over = True
```

Leave that completely alone.

6. Add ammo to the HUD

You previously changed the HUD for Task 1 so that it contains HP.

For example, you should now have something like:

`</>` Python

```
f"HP: {self.player.hp} Wave: {self.wave} Score: ..."
```

Add ammo immediately after HP:

`</>` Python

```
f"HP: {self.player.hp} Ammo: {self.player.ammo} Wave: {self.wave} Score: ..."
```

So your existing HUD line should become approximately:

`</>` Python

```
hud = self.font.render(  
    f"HP: {self.player.hp} Ammo: {self.player.ammo} Wave: {self.wave} Score: {self.score} Kills: {self.kills}  
    True, (160,220,120))
```

That's the **only HUD change** required.

7. What you should NOT change

For Task 2, don't modify:

- Zombie
- `spawn_zombie()`
- zombie HP
- zombie speed
- player HP
- player invincibility
- player movement
- bullet movement

- bullet collision
- score
- kills
- waves
- game-over logic
- restart logic
- graphics other than the HUD

In particular, **don't rewrite `Player.shoot()` completely**. Just add the ammo/reload checks and the two ammo operations.

Final Task 2 changes

You should end up making changes in only these places:

Location	Task 2 change
<code>Player.__init__()</code>	<code>ammo = 12 , reload_timer = 0</code>
<code>Player.move()</code>	Countdown reload timer
<code>Player.shoot()</code>	Prevent shooting during reload/empty clip
<code>Player.shoot()</code>	Decrease ammo after successful shot
<code>Player.shoot()</code>	Start 120-frame reload at 0 ammo
<code>GameEngine.draw()</code>	Display ammo

That's all.

Test Task 2 before committing

Run these tests:

Test 1 — Initial ammo

HUD:

</> Plain text

HP: 3 Ammo: 12

Test 2 — Shooting

Shoot once:

</> Plain text

Ammo: 11

Shoot again:

</> Plain text

Ammo: 10

Continue until:

</> Plain text

Ammo: 0

Test 3 — Empty clip

At:

</> Plain text

Ammo: 0

click again.

No bullet should be created.

Test 4 — Reload

At zero ammo:

</> Plain text

Ammo: 0

Wait exactly **2 seconds**.

Then:

 Plain text

Ammo: 12

Test 5 — Health regression

While testing Task 2, deliberately let a zombie touch you.

Your existing Task 1 behavior should still be:

 Plain text

```
3 HP
↓
2 HP
↓
invincibility
```

If the Health System behaves differently, don't fix it as part of Task 2. That would mix tasks.

Once all of this passes, make the separate Task 2 commit:

 Bash

```
git add game.py
git commit -m "Task 2: Add ammo and reload system"
```

This keeps your required **Task 1 commit separate from Task 2**, as the lab requires.

 Lab_4_VibeCoding_Student_handout

Keep Tasks 1 and 2 exactly as they are and implement Task 3 only. Place 4 explosive barrels on the map. When a bullet hits a barrel, trigger an explosion that destroys all zombies within a fixed radius. Remove the barrel after it explodes. Keep the implementation simple, preserve the existing code structure, and make only the changes required by the task.

Yes. Keep **Task 1 (Health)** and **Task 2 (Ammo/Reload)** completely unchanged.

For **Task 3 only**, make the following small changes to your current `game.py`. The existing bullet collision system can be reused, so there is no need to redesign it.

1. Add a simple `Barrel` class

Place this **after the `Zombie` class and before `spawn_zombie()`**.

`</>` Python

```
class Barrel:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 30, 30)
        self.explosion_radius = 100

    def draw(self, screen):
        pygame.draw.rect(screen, (150, 80, 30), self.rect)
        pygame.draw.circle(screen, (220, 180, 50), self.rect.center, 8)
```

This gives us:

- A simple 30×30 barrel
- A fixed explosion radius of 100 pixels
- Simple visual representation

No external code or new dependency is needed.

2. Create 4 barrels in `GameEngine.reset()`

Find your existing `reset()` method.

You already have something similar to:

`</>` Python

```
self.zombies = []
```

Immediately after that, add:

`</>` Python

```
self.barrels = [
    Barrel(150, 150),
    Barrel(WIDTH - 180, 150),
```

```
        Barrel(150, HEIGHT - 180),  
        Barrel(WIDTH - 180, HEIGHT - 180)  
    ]
```

So you will have exactly **4 barrels**.

Don't change your existing zombie, player, score, wave, HP, or ammo initialization.

3. Draw the barrels

Go to `GameEngine.draw()` .

You already draw the zombies and player there.

Add:

`</>` Python

```
for barrel in self.barrels:  
    barrel.draw(screen)
```

Put it before or after the zombie drawing; either is fine for this simple task.

For example:

`</>` Python

```
for barrel in self.barrels:  
    barrel.draw(screen)  
  
for z in self.zombies:  
    z.draw(screen)  
  
self.player.draw(screen)
```

Do **not** modify your existing HP/Ammo HUD.

Your Task 1/2 HUD should remain exactly as it is.

4. Add barrel collision to the existing bullet logic

This is the most important change.

Go to `GameEngine.update()` and find your **existing bullet/zombie collision section**.

It will be the part where bullets are checked against zombies and zombies lose HP.

You should **not replace that system**.

Inside the existing bullet processing, after you have the bullet's current position, add a barrel check.

Conceptually, for each bullet:

`</>` Python

```
for barrel in self.barrels:
    if barrel.rect.collidepoint(int(bullet[0]), int(bullet[1])):
```

When a barrel is hit, destroy nearby zombies.

Add:

`</>` Python

```
for z in self.zombies:
    dx = z.rect.centerx - barrel.rect.centerx
    dy = z.rect.centery - barrel.rect.centery
    if dx * dx + dy * dy <= barrel.explosion_radius ** 2:
        z.hp = 0
```

Then remove the barrel:

`</>` Python

```
self.barrels.remove(barrel)
```

And remove the bullet that caused the explosion.

Important: use your existing bullet variable

Your current project stores bullets as:

```
</> Python
```

```
[x, y, vx, vy]
```

So don't change the bullet representation.

If your current code uses a variable such as:

```
</> Python
```

```
b
```

instead of `bullet` , use that existing variable.

For example, if your current loop is:

```
</> Python
```

```
for b in self.player.bullets[:]:
```

then use:

```
</> Python
```

```
if barrel.rect.collidepoint(int(b[0]), int(b[1])):
```

rather than introducing another bullet representation.

5. One small but important detail

Because you are modifying a list while iterating through it, use a copy when checking the barrels:

```
</> Python
```

```
for barrel in self.barrels[:]:
```

Then:

```
</> Python
```

```
if barrel.rect.collidepoint(int(b[0]), int(b[1])):
    for z in self.zombies:
        dx = z.rect.centerx - barrel.rect.centerx
        dy = z.rect.centery - barrel.rect.centery

        if dx * dx + dy * dy <= barrel.explosion_radius ** 2:
            z.hp = 0

    self.barrels.remove(barrel)
    self.player.bullets.remove(b)
    break
```

The `break` prevents the same bullet from being processed against another barrel after it has already exploded one.

6. Do NOT change the existing zombie kill system

This is important.

Your existing code already handles zombie HP and removes dead zombies.

So don't write something like:

</> Python

```
self.zombies.remove(z)
```

inside the explosion code.

Instead:

</> Python

```
z.hp = 0
```

Then let your **existing zombie/bullet processing logic** handle removal and kill counting.

This preserves:

- Existing score
- Existing kills

- Existing wave progression
- Existing zombie HP behavior

and keeps Task 3 isolated.

What you should NOT touch

For this task, leave all of these exactly as they are:

Task 1

 Python

```
self.hp = 3
self.invincibility_timer = ...
```

and your zombie collision/health logic.

Task 2

 Python

```
self.ammo = 12
self.reload_timer = ...
```

and all shooting/reload logic.

Also don't modify:

- Zombie
- `spawn_zombie()`
- player movement
- bullet speed
- shooting cooldown
- HP HUD
- Ammo HUD
- score calculation
- wave generation
- game-over logic

Task 3 changes summary

You should only have changes in these locations:

Location	Change
After <code>Zombie</code> class	Add <code>Barrel</code> class
<code>GameEngine.reset()</code>	Create 4 barrels
<code>GameEngine.update()</code>	Detect bullet → barrel collision and explosion
<code>GameEngine.draw()</code>	Draw barrels

That's it.

Testing

After implementing, test specifically:

1. Start game → **4 barrels visible**.
2. Shoot a barrel.
3. Barrel disappears.
4. Zombies within the **100-pixel radius** are destroyed.
5. Zombies outside the radius survive.
6. Shoot another barrel → it also explodes.
7. Verify Task 1 still works:
 - HP decreases on zombie contact.
 - Invincibility still works.
8. Verify Task 2 still works:
 - Ammo decreases.
 - Empty clip reloads after 2 seconds.

Once Task 3 passes, commit **only Task 3**:

`</>` Bash

```
git add game.py
git commit -m "Task 3: Add explosive barrels"
```

That gives you the required separate Task 3 commit while leaving Tasks 1 and 2 intact.

 Lab_4_VibeCoding_Student_handout

Keep Tasks 1, 2, and 3 unchanged and implement Task 4. Add a Fast Zombie with higher speed, 1 HP, and smaller size, and a Tank Zombie with lower speed, 6 HP, and larger size. Mix both types into wave spawning alongside the standard zombie. Make sure wave progression still functions. Preserve the existing code structure and do not add unnecessary features. If the Tank's lower fractional speed causes movement to stop because of integer position updates, fix only that movement issue. Finally, make sure every new wave has 4 explosive barrels again, while keeping the original Task 3 barrel behavior unchanged.

Yes. **Do not change anything from Tasks 1–3.** For Task 4, make only the following targeted changes.

One important addition from your requirement: **when a new wave starts, recreate the 4 barrels**, but keep the existing Task 3 barrel explosion behavior exactly as it is.

1. Modify the `Zombie` class to support types

Find your existing `Zombie.__init__()` .

It currently has the standard zombie properties such as position, speed, HP, and size.

Change the constructor so it accepts a type:

 Python

```
def __init__(self, x, y, zombie_type="standard"):
```

Then keep the standard values as the default, and add the two new types.

Use this logic inside `__init__()` :

 Python

```
self.type = zombie_type
```

```
if zombie_type == "fast":
```

```
        self.speed = 3.0
        self.hp = 1
        self.size = 20
    elif zombie_type == "tank":
        self.speed = 0.75
        self.hp = 6
        self.size = 40
    else:
        self.speed = 1.5
        self.hp = 3
        self.size = 30

    self.rect = pygame.Rect(x, y, self.size, self.size)
```

Why these values?

The requirement specifies relative behavior:

- Fast → higher speed, 1 HP, smaller
- Tank → lower speed, 6 HP, larger
- Standard → existing values

So:

</> Plain text

```
Standard: 1.5 speed / 3 HP / 30
Fast:      3.0 speed / 1 HP / 20
Tank:      0.75 speed / 6 HP / 40
```

This keeps the original standard zombie unchanged.

2. Fix the Tank movement issue only

This is important because your existing movement likely does something equivalent to:

</> Python

```
self.rect.x += int(dx / dist * self.speed)
self.rect.y += int(dy / dist * self.speed)
```

With Tank speed 0.75, the fractional movement gets converted to:

Plain text

```
int(0.75 * direction)
→ 0
```

so the tank can appear stationary.

Don't redesign movement.

Change only the position update so the fractional movement isn't discarded.

The simplest approach is to give the zombie floating-point coordinates.

In `Zombie.__init__()` , after creating the rectangle:

Python

```
self.x = float(self.rect.x)
self.y = float(self.rect.y)
```

Then in `Zombie.update()` , instead of directly incrementing the integer rectangle coordinates, update `self.x` and `self.y` .

For example, if your existing code has:

Python

```
self.rect.x += int(dx / dist * self.speed)
self.rect.y += int(dy / dist * self.speed)
```

change **only those two lines** to:

Python

```
self.x += dx / dist * self.speed
self.y += dy / dist * self.speed

self.rect.x = int(self.x)
self.rect.y = int(self.y)
```

This fixes the fractional-speed problem while preserving the existing movement algorithm.

Do not change the movement formula itself.

3. Modify `spawn_zombie()`

Your existing `spawn_zombie()` creates a standard zombie.

Keep its existing spawn-position logic.

The only thing you need to add is choosing a zombie type.

For example, if the current function ends with something like:

`</>` Python

```
return Zombie(x, y)
```

change that to:

`</>` Python

```
zombie_type = random.choices(  
    ["standard", "fast", "tank"],  
    weights=[6, 2, 1]  
)[0]  
  
return Zombie(x, y, zombie_type)
```

This means every newly spawned zombie can be:

`</>` Plain text

```
Standard  
Fast  
Tank
```

with standard zombies remaining the most common.

You don't need a complicated wave-specific spawning system.

4. Preserve wave progression

Do not modify your existing wave calculation unnecessarily.

If your current code does something like:

`</>` Python

```
if self.kills >= self.kills_to_next:
    self.wave += 1
    self.kills_to_next += ...
...
```

leave that logic intact.

The new zombie types are still instances of `Zombie` , so the existing:

`</>` Python

```
z.update(...)
z.hit()
z.draw(...)
```

logic continues to work.

That is exactly what we want.

5. Make 4 barrels appear at every new wave

This is the one Task 3-related change required by Task 4.

You already created:

`</>` Python

```
self.barrels = [
    Barrel(...),
    Barrel(...),
    Barrel(...),
    Barrel(...)
]
```

inside `reset()` .

Do not change the existing barrel class or explosion code.

Instead, when your existing wave progression detects a new wave, recreate the same four barrels.

The cleanest way is to add a small helper to `GameEngine` :

`</>` Python

```
def reset_barrels(self):
    self.barrels = [
        Barrel(150, 150),
        Barrel(WIDTH - 180, 150),
        Barrel(150, HEIGHT - 180),
        Barrel(WIDTH - 180, HEIGHT - 180)
    ]
```

Then in `reset()` replace the four-line barrel initialization with:

`</>` Python

```
self.reset_barrels()
```

This doesn't change the barrel behavior; it just avoids duplicating the coordinates.

6. Call `reset_barrels()` when a new wave starts

Find the existing code that increments the wave.

For example:

`</>` Python

```
self.wave += 1
```

Immediately after the wave increment, add:

`</>` Python

```
self.reset_barrels()
```

So conceptually:

</> Python

```
self.wave += 1
self.reset_barrels()
```

Leave the rest of the wave logic alone.

Now the behavior becomes:

</> Plain text

```
Wave 1
├─ 4 barrels
├─ Standard/Fast/Tank zombies
└─ play
```

```
All zombies cleared
```

```
↓
```

```
Wave 2
├─ 4 NEW barrels
├─ Standard/Fast/Tank zombies
└─ play
```

```
All zombies cleared
```

```
↓
```

```
Wave 3
├─ 4 NEW barrels
└─ ...
```

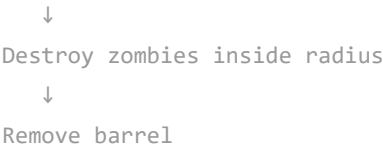
If a player destroys 3 barrels during Wave 1, only one remains. When Wave 2 begins, the four barrels are recreated.

7. Don't touch your Task 3 explosion code

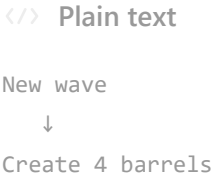
Keep this behavior exactly as you implemented it:

</> Plain text

```
Bullet
↓
Barrel collision
↓
Explosion radius
```



The only new behavior is:



So don't change:

- explosion radius
- barrel collision
- zombie destruction
- barrel removal
- bullet behavior

8. Your Task 4 changes should therefore be limited to

Location	Change
Zombie.__init__()	Add zombie type and type-specific speed/HP/size
Zombie.__init__()	Add floating-point x/y positions
Zombie.update()	Preserve movement formula but retain fractional movement
spawn_zombie()	Randomly choose Standard/Fast/Tank
GameEngine	Add reset_barrels() helper
Wave progression	Call reset_barrels() at the start of each new wave

That's all.

9. Do NOT modify Tasks 1–3

Specifically leave your existing:

Health System

`</>` Python

```
self.hp = 3
self.invincibility_timer = ...
```

and collision/invincibility logic unchanged.

Ammo System

`</>` Python

```
self.ammo = 12
self.reload_timer = ...
```

and your 2-second reload logic unchanged.

Explosive Barrels

Leave the existing `Barrel` class and `bullet` → `barrel` → explosion behavior unchanged.

10. Testing Task 4

Run these tests before committing.

Standard zombie

Verify it still has:

`</>` Plain text

```
Speed = 1.5
HP = 3
Size = 30×30
```

Fast zombie

Verify:

`</>` Plain text

```
Speed > standard
HP = 1
Size < standard
```

Shoot it once → it should die.

Tank zombie

Verify:

`</>` Plain text

```
Speed < standard
HP = 6
Size > standard
```

Shoot it repeatedly → it should require **6 HP worth of damage**.

Most importantly, verify the Tank actually moves. If it was previously stopping because of integer truncation, the floating-point position fix should make it move smoothly.

Wave progression

Clear the required zombies and verify:

`</>` Plain text

```
Wave 1 → Wave 2 → Wave 3 ...
```

continues normally.

Barrels

At the beginning of each wave:

`</>` Plain text

```
4 barrels
```

Shoot one:

 Plain text

4 → 3

Then start the next wave:

 Plain text

3 → 4

And verify the **original explosion behavior still works**.

11. Task 4 commit

Once all of that passes:

 Bash

```
git add game.py
git commit -m "Task 4: Add fast and tank zombies"
```

This keeps Tasks 1, 2, 3, and 4 as separate commits, exactly as required by the lab.

 Lab_4_VibeCoding_Student_handout

One caution: don't add extra features such as zombie health bars, explosion animations, new sounds, new HUD elements, difficulty scaling, or special Fast/Tank graphics. None of those are required for Task 4, and adding them would make the change larger than necessary.